

Innopolis University

Database Course

Database Assignment Report

Assignment 2: SQL & Optimization

Team #68

April 2026

1 Team Contributions

Name	Email	Contribution
Ilia Kazachkov	i.kazachkov	Consolidated the deliverables and prepared the final report.
Ilya Muhin	i.muhin	Designed and implemented the indexing strategy used for query optimization.
Ildar Mukhamadullin	i.mukhamadullin	Developed the SQL solutions for Query 1, Query 2, and Query 3.
Irina Perekrestova	i.perekrestova	Prepared and integrated the data insertion part of the hospital population script.

2 Team Meetings

Date	Summary
Mar, 31 2026	Roles assigned.
Apr, 5 2026	Inserts were made and discussed.
Apr, 6 2026	Queries and indexes created, task fully implemented.
Apr, 7 2026	Assignment report made and submitted.

3 Overview

This report documents the optimization stage of the hospital database assignment for Team #68. The submitted solution includes:

- a creational and population script for the hospital database,
- three analytical SQL queries,
- an index script aimed at reducing query execution cost,
- and this report summarizing the effect of the proposed indexes.

The population script imports a non-trivial dataset derived from the provided CSV files. In the final script, the core entity tables `DEPARTMENT`, `DOCTOR`, `PATIENT`, `PROCEDURE`, `APPOINTMENT`, and `DIAGNOSIS` each contain 1000 inserted rows, while `APPOINTMENT_PROCEDURE` contains 1500 inserted rows. This data volume is large enough for query planning choices and index usage to matter in practice.

4 Indexing Strategy

The following indexes were added in `team_68_indexes.sql`:

- `idx_doctor_department_id` on `DOCTOR(Department_ID)`
- `idx_appointment_doctor_id` on `APPOINTMENT(Doctor_ID)`
- `idx_appointment_patient_id` on `APPOINTMENT(Patient_ID)`
- `idx_appointment_patient_date` on `APPOINTMENT(Patient_ID, Date)`
- `idx_appointment_status` on `APPOINTMENT(Status)`
- `idx_diagnosis_appointment_id` on `DIAGNOSIS(Appointment_ID)`
- `idx_appt_proc_appt_id` on `APPOINTMENT_PROCEDURE(Appointment_ID)`
- `idx_appt_proc_proc_id` on `APPOINTMENT_PROCEDURE(Procedure_ID)`
- `idx_procedure_id_price` on `PROCEDURE(ID, Price)`
- `idx_patient_id` on `PATIENT(ID)`

In general, the chosen indexes target the most expensive parts of the workload: joins across foreign keys, aggregation over appointments, and the self-join pattern used in Query 3. The expected impact is not identical across all queries. Some indexes directly match join predicates and ordering patterns, while others serve more as supporting indexes and may have limited influence depending on the planner's chosen execution strategy.

5 Query 1 Optimization

5.1 Query Goal

Query 1 returns doctors who handled more than 20 appointments and reports total, completed, and cancelled appointment counts together with the department name.

5.2 Indexes Most Relevant to Query 1

The most relevant index for this query is `idx_appointment_doctor_id`, because appointments are grouped per doctor after joining `APPOINTMENT` with `DOCTOR`. The index `idx_appointment_status` may provide limited support, although its effect is usually smaller because `Status` is used inside filtered aggregates rather than in a selective `WHERE` clause.

5.3 EXPLAIN Before Applying Indexes

Listing 1: EXPLAIN output for Query 1 before indexes

```
"Sort (cost=129.39..130.10 rows=286 width=96)"
"  Sort Key: (count(a.id)) DESC, d.full_name"
"  -> HashAggregate (cost=106.99..117.72 rows=286 width=96)"
```

```

"      Group Key: d.id, dep.name"
"      Filter: (count(a.id) > 20)"
"      -> Hash Join (cost=67.87..91.98 rows=858 width=112)"
"          Hash Cond: (d.department_id = dep.id)"
"      -> Hash Join (cost=33.80..55.64 rows=858 width=88)"
"          Hash Cond: (a.doctor_id = d.id)"
"      -> Seq Scan on appointment a (cost=0.00..19.58 rows=858
width=48)"
"          -> Hash (cost=25.02..25.02 rows=702 width=48)"
"              -> Seq Scan on doctor d (cost=0.00..25.02 rows=702
width=48)"
"      -> Hash (cost=20.70..20.70 rows=1070 width=40)"
"      -> Seq Scan on department dep (cost=0.00..20.70 rows=1070
width=40)"

```

5.4 EXPLAIN After Applying Indexes

Listing 2: EXPLAIN output for Query 1 after indexes

```

"Sort (cost=142.22..143.06 rows=333 width=66)"
"  Sort Key: (count(a.id)) DESC, d.full_name"
"  -> HashAggregate (cost=115.77..128.27 rows=333 width=66)"
"      Group Key: d.id, dep.name"
"      Filter: (count(a.id) > 20)"
"      -> Hash Join (cost=72.00..98.27 rows=1000 width=59)"
"          Hash Cond: (d.department_id = dep.id)"
"      -> Hash Join (cost=40.50..64.14 rows=1000 width=47)"
"          Hash Cond: (a.doctor_id = d.id)"
"      -> Seq Scan on appointment a (cost=0.00..21.00 rows=1000
width=25)"
"          -> Hash (cost=28.00..28.00 rows=1000 width=30)"
"              -> Seq Scan on doctor d (cost=0.00..28.00 rows=1000
width=30)"
"      -> Hash (cost=19.00..19.00 rows=1000 width=28)"
"      -> Seq Scan on department dep (cost=0.00..19.00 rows=1000
width=28)"

```

5.5 Discussion

Based on the collected EXPLAIN output, Query 1 did not show an improvement after applying the indexes. The top estimated sort cost increased from 129.39..130.10 before indexing to 142.22..143.06 after indexing. In both plans, PostgreSQL still chose sequential scans on `APPOINTMENT`, `DOCTOR`, and `DEPARTMENT`, followed by hash joins and hash aggregation.

This result is reasonable for an aggregation-heavy query that processes a large share of the available rows. The index on `APPOINTMENT(Doctor_ID)` is relevant to the join

condition, but the planner estimated that scanning and aggregating the tables directly was still cheaper than switching to index-driven access. The index on **Status** also had little influence, since **Status** is used only inside filtered aggregates and not in a selective **WHERE** clause.

Therefore, for the current dataset and query shape, the added indexes did not reduce the estimated execution cost of Query 1. The most likely reason is that the query remains dominated by grouping and aggregation rather than by selective row lookup.

6 Query 2 Optimization

6.1 Query Goal

Query 2 finds patients who visited at least three distinct departments and whose total spending on procedures is greater than the average spending across patients. The query combines multiple joins, aggregations, and a comparison against an aggregated subquery.

6.2 Indexes Most Relevant to Query 2

This query is supported mainly by the following indexes:

- `idx_appointment_patient_id` for grouping and joining appointments per patient,
- `idx_diagnosis_appointment_id` for counting diagnoses by appointment,
- `idx_appt_proc_appt_id` and `idx_appt_proc_proc_id` for procedure-cost aggregation,
- `idx_doctor_department_id` as supporting infrastructure for the doctor-to-department relationship.

6.3 EXPLAIN Before Applying Indexes

Listing 3: EXPLAIN output for Query 2 before indexes

```
"Sort (cost=515.11..516.50 rows=554 width=78)"
"  Sort Key: sp.total_spent DESC, (count(DISTINCT a.id)) DESC, p.full_name"
"  CTE patient_spending"
"    -> HashAggregate (cost=147.88..160.38 rows=1000 width=40)"
"          Group Key: p_1.id"
"          -> Hash Left Join (cost=103.50..140.38 rows=1500 width=14)"
"                Hash Cond: (ap.procedure_id = pr.id)"
"                -> Hash Right Join (cost=73.00..105.92 rows=1500 width=16)"
"                      Hash Cond: (a_1.patient_id = p_1.id)"
"                      -> Hash Right Join (cost=33.50..62.46 rows=1500 width
=16)"
"                                Hash Cond: (ap.appointment_id = a_1.id)"
"                                -> Seq Scan on appointment_procedure ap (cost
=0.00..25.00 rows=1500 width=16)"
```

```

"          -> Hash (cost=21.00..21.00 rows=1000 width=16)"
"          -> Seq Scan on appointment a_1 (cost
=0.00..21.00 rows=1000 width=16)"
"          -> Hash (cost=27.00..27.00 rows=1000 width=8)"
"          -> Seq Scan on patient p_1 (cost=0.00..27.00 rows
=1000 width=8)"
"          -> Hash (cost=18.00..18.00 rows=1000 width=14)"
"          -> Seq Scan on procedure pr (cost=0.00..18.00 rows=1000
width=14)"
" InitPlan 2"
"   -> Aggregate (cost=23.34..23.35 rows=1 width=32)"
"       -> CTE Scan on patient_spending (cost=0.00..22.50 rows=333 width
=32)"
"           Filter: (total_spent > '0'::numeric)"
"   -> Hash Join (cost=261.04..306.14 rows=554 width=78)"
"       Hash Cond: (p.id = sp.patient_id)"
"       -> GroupAggregate (cost=234.37..259.37 rows=333 width=46)"
"           Group Key: p.id"
"           Filter: (count(DISTINCT d.id) >= 3)"
"       -> Sort (cost=234.37..236.87 rows=1000 width=46)"
"           Sort Key: p.id, d.id"
"       -> Hash Left Join (cost=145.00..184.54 rows=1000 width
=46)"
"           Hash Cond: (doc.department_id = d.id)"
"       -> Hash Left Join (cost=113.50..150.41 rows=1000
width=46)"
"           Hash Cond: (a.doctor_id = doc.id)"
"       -> Hash Right Join (cost=73.00..107.27 rows
=1000 width=46)"
"           Hash Cond: (a.patient_id = p.id)"
"       -> Hash Right Join (cost=33.50..65.14
rows=1000 width=32)"
"           Hash Cond: (diag.appointment_id = a
.id)"
"       -> Seq Scan on diagnosis diag (
cost=0.00..29.00 rows=1000 width=16)"
"       -> Hash (cost=21.00..21.00 rows
=1000 width=24)"
"           -> Seq Scan on appointment a
(cost=0.00..21.00 rows=1000 width=24)"
"       -> Hash (cost=27.00..27.00 rows=1000
width=22)"
"           -> Seq Scan on patient p (cost
=0.00..27.00 rows=1000 width=22)"
"       -> Hash (cost=28.00..28.00 rows=1000 width
=16)"

```

```

"                                -> Seq Scan on doctor doc  (cost
=0.00..28.00 rows=1000 width=16)"
"                                -> Hash  (cost=19.00..19.00 rows=1000 width=8)"
"                                -> Seq Scan on department d  (cost=0.00..19.00
rows=1000 width=8)"
"                                -> Hash  (cost=22.50..22.50 rows=333 width=40)"
"                                -> CTE Scan on patient_spending sp  (cost=0.00..22.50 rows=333
width=40)"
"                                Filter: (total_spent > (InitPlan 2).col1)".

```

6.4 EXPLAIN After Applying Indexes

Listing 4: EXPLAIN output for Query 2 after indexes

```

"Sort  (cost=515.11..516.50 rows=554 width=78)"
"  Sort Key: sp.total_spent DESC, (count(DISTINCT a.id)) DESC, p.full_name"
"  CTE patient_spending"
"    -> HashAggregate  (cost=147.88..160.38 rows=1000 width=40)"
"      Group Key: p_1.id"
"      -> Hash Left Join  (cost=103.50..140.38 rows=1500 width=14)"
"        Hash Cond: (ap.procedure_id = pr.id)"
"        -> Hash Right Join  (cost=73.00..105.92 rows=1500 width=16)"
"          Hash Cond: (a_1.patient_id = p_1.id)"
"          -> Hash Right Join  (cost=33.50..62.46 rows=1500 width
=16)"
"            Hash Cond: (ap.appointment_id = a_1.id)"
"            -> Seq Scan on appointment_procedure ap  (cost
=0.00..25.00 rows=1500 width=16)"
"              -> Hash  (cost=21.00..21.00 rows=1000 width=16)"
"                -> Seq Scan on appointment a_1  (cost
=0.00..21.00 rows=1000 width=16)"
"                  -> Hash  (cost=27.00..27.00 rows=1000 width=8)"
"                    -> Seq Scan on patient p_1  (cost=0.00..27.00 rows
=1000 width=8)"
"                      -> Hash  (cost=18.00..18.00 rows=1000 width=14)"
"                        -> Seq Scan on procedure pr  (cost=0.00..18.00 rows=1000
width=14)"
"  InitPlan 2"
"    -> Aggregate  (cost=23.34..23.35 rows=1 width=32)"
"      -> CTE Scan on patient_spending  (cost=0.00..22.50 rows=333 width
=32)"
"        Filter: (total_spent > '0'::numeric)"
"    -> Hash Join  (cost=261.04..306.14 rows=554 width=78)"
"      Hash Cond: (p.id = sp.patient_id)"
"      -> GroupAggregate  (cost=234.37..259.37 rows=333 width=46)"
"        Group Key: p.id"
"        Filter: (count(DISTINCT d.id) >= 3)"

```

```

"          -> Sort (cost=234.37..236.87 rows=1000 width=46)"
"              Sort Key: p.id, d.id"
"          -> Hash Left Join (cost=145.00..184.54 rows=1000 width
=46)"
"              Hash Cond: (doc.department_id = d.id)"
"          -> Hash Left Join (cost=113.50..150.41 rows=1000
width=46)"
"              Hash Cond: (a.doctor_id = doc.id)"
"          -> Hash Right Join (cost=73.00..107.27 rows
=1000 width=46)"
"              Hash Cond: (a.patient_id = p.id)"
"          -> Hash Right Join (cost=33.50..65.14
rows=1000 width=32)"
"              Hash Cond: (diag.appointment_id = a
.id)"
"          -> Seq Scan on diagnosis diag (
cost=0.00..29.00 rows=1000 width=16)"
"          -> Hash (cost=21.00..21.00 rows
=1000 width=24)"
"              -> Seq Scan on appointment a
(cost=0.00..21.00 rows=1000 width=24)"
"          -> Hash (cost=27.00..27.00 rows=1000
width=22)"
"              -> Seq Scan on patient p (cost
=0.00..27.00 rows=1000 width=22)"
"          -> Hash (cost=28.00..28.00 rows=1000 width
=16)"
"              -> Seq Scan on doctor doc (cost
=0.00..28.00 rows=1000 width=16)"
"          -> Hash (cost=19.00..19.00 rows=1000 width=8)"
"          -> Seq Scan on department d (cost=0.00..19.00
rows=1000 width=8)"
"      -> Hash (cost=22.50..22.50 rows=333 width=40)"
"      -> CTE Scan on patient_spending sp (cost=0.00..22.50 rows=333
width=40)"
"      Filter: (total_spent > (InitPlan 2).col1)"

```

6.5 Discussion

For Query 2, the EXPLAIN output before and after indexing is effectively the same. The top estimated cost remains 515.11..516.50, and the planner keeps the same overall strategy based on hash joins, sequential scans, CTE processing, sorting, and group aggregation. No visible index scan or bitmap scan was introduced in the posted plan.

This means that, on the current dataset, the proposed indexes did not produce a measurable improvement in the planner's estimated execution cost. Although several of the added indexes match foreign-key join columns used by the query, the statement

still builds large intermediate results and performs multiple aggregations. Under such conditions, PostgreSQL may prefer full scans and hash-based processing over index access.

A likely explanation is that the query is dominated by aggregation and broad joins rather than by highly selective predicates. As a result, the optimizer estimated that the existing plan remained the cheapest one even after the supporting indexes were created.

7 Query 3 Optimization

7.1 Query Goal

Query 3 returns appointments for which the same patient had at least three earlier appointments on strictly earlier dates. This is implemented through a self-join on `APPOINTMENT` using patient identity and date comparison.

7.2 Indexes Most Relevant to Query 3

The key index for this query is `idx_appointment_patient_date` on `APPOINTMENT(Patient_ID, Date)`. This index matches the self-join predicate very closely, because the query compares rows using both `Patient_ID` and `Date`. The index `idx_appointment_patient_id` may also provide auxiliary benefit, but the composite index is the main optimization mechanism here.

7.3 EXPLAIN Before Applying Indexes

Listing 5: EXPLAIN output for Query 3 before indexes

```
"Sort (cost=164.64..164.92 rows=111 width=56)"
"  Sort Key: p.full_name, a2.date, a2."time""
"  -> HashAggregate (cost=156.71..160.87 rows=111 width=56)"
"    Group Key: p.full_name, a2.id, d.full_name"
"    Filter: (count(a1.id) >= 3)"
"    -> Hash Join (cost=117.05..153.38 rows=333 width=56)"
"      Hash Cond: (d.id = a2.doctor_id)"
"      -> Seq Scan on doctor d (cost=0.00..28.00 rows=1000 width=22)"
"      -> Hash (cost=112.89..112.89 rows=333 width=50)"
"        -> Hash Join (cost=75.64..112.89 rows=333 width=50)"
"          Hash Cond: (a2.patient_id = p.id)"
"          Join Filter: (a1.date < a2.date)"
"          -> Seq Scan on appointment a2 (cost=0.00..21.00
rows=1000 width=36)"
"            -> Hash (cost=63.14..63.14 rows=1000 width=42)"
"              -> Hash Join (cost=39.50..63.14 rows=1000
width=42)"
"                Hash Cond: (a1.patient_id = p.id)"
"                -> Seq Scan on appointment a1 (cost
=0.00..21.00 rows=1000 width=20)"
```

```

"                                -> Hash  (cost=27.00..27.00 rows=1000
width=22)"
"                                -> Seq Scan on patient p  (cost
=0.00..27.00 rows=1000 width=22)"

```

7.4 EXPLAIN After Applying Indexes

Listing 6: EXPLAIN output for Query 3 after indexes

```

"Sort  (cost=164.64..164.92 rows=111 width=56)"
"  Sort Key: p.full_name, a2.date, a2.""time""
"  -> HashAggregate  (cost=156.71..160.87 rows=111 width=56)"
"    Group Key: p.full_name, a2.id, d.full_name"
"    Filter: (count(a1.id) >= 3)"
"    -> Hash Join  (cost=117.05..153.38 rows=333 width=56)"
"      Hash Cond: (d.id = a2.doctor_id)"
"      -> Seq Scan on doctor d  (cost=0.00..28.00 rows=1000 width=22)"
"      -> Hash  (cost=112.89..112.89 rows=333 width=50)"
"        -> Hash Join  (cost=75.64..112.89 rows=333 width=50)"
"          Hash Cond: (a2.patient_id = p.id)"
"          Join Filter: (a1.date < a2.date)"
"          -> Seq Scan on appointment a2  (cost=0.00..21.00
rows=1000 width=36)"
"            -> Hash  (cost=63.14..63.14 rows=1000 width=42)"
"              -> Hash Join  (cost=39.50..63.14 rows=1000
width=42)"
"                Hash Cond: (a1.patient_id = p.id)"
"                -> Seq Scan on appointment a1  (cost
=0.00..21.00 rows=1000 width=20)"
"                  -> Hash  (cost=27.00..27.00 rows=1000
width=22)"
"                    -> Seq Scan on patient p  (cost
=0.00..27.00 rows=1000 width=22)"

```

7.5 Discussion

For Query 3, the EXPLAIN output also shows no visible change after indexing. The top estimated cost remains 164.64..164.92, and the posted plan keeps the same structure: sequential scans on the **APPOINTMENT** table aliases, hash joins, and a final hash aggregate with sorting.

This indicates that the composite index on **APPOINTMENT(Patient_ID, Date)** did not change the planner's chosen strategy for the current dataset. Even though this index is well aligned with the self-join predicate, the optimizer still estimated that scanning the tables and applying the join/filter logic in memory was cheaper than using index-based access paths.

The most probable reason is the size and distribution of the data. When a large portion of rows is still expected to participate in the join, PostgreSQL may continue to prefer sequential scans. Therefore, for the current test data, Query 3 did not achieve a lower estimated cost despite having a logically relevant composite index.

8 Conclusion

The indexing strategy was designed to support the joins and access patterns used by the three required queries. However, the collected `EXPLAIN` results show that the actual impact on the planner was limited for this dataset.

In summary:

- Query 1 did not improve according to the posted execution plan. Its top estimated cost increased after indexing, and PostgreSQL still preferred sequential scans, hash joins, and aggregation.
- Query 2 showed no visible change in estimated cost or plan structure before and after indexing. The query remained dominated by CTE processing, multiple joins, and aggregation over large intermediate results.
- Query 3 also showed no visible change in estimated cost or execution strategy. Even though the composite index on `APPOINTMENT(Patient_ID, Date)` is logically relevant, PostgreSQL continued to use sequential scans for the current dataset.

Overall, the selected indexes are valid and aligned with the query predicates, but their effect was limited in the observed execution plans. This outcome is still consistent with PostgreSQL optimization behavior: when queries touch a large fraction of the rows and are dominated by grouping or hash-based joins, the planner may decide that sequential scans remain more efficient than index access.

9 Use of AI Tools

AI tools were used to improve the phrasing and formatting of the final report. The SQL scripts, indexing choices, and submitted deliverables were prepared and reviewed by the team members.